Lycée Secondaire Hammam-Sousse	Devoir de Synthèse n°2	**Matière : Informatique** **Classe :** 2TI - **Durée :** 2h **Enseignants :** M MANI, Mme Achour, Mme Chtioui **Année Scolaire :** 2024-2025

Nom & Prénom : .....

Note / 20

Exercice 1 (4 Points)

Soit l'algorithme du module **Construction** :

```
..................................... Construction(ch: Chaîne):.....................................
Début
  C ← ""
  V ← ""
  Pour i de 0 à Long(ch) - 1 Faire
    Si "A" ≤ ch[i] ≤ "Z" Alors
      Si Pos(ch[i], "OIYEAU") = -1 Alors
        C ← C + ch[i]
      Sinon
        V ← V + ch[i]
      Fin Si
    Fin Si
  Fin Pour
  Retourner C + "|" + V
Fin
```

T.D.O.L

Objet	Type

1) Compléter l'entête du module **Construction**, ainsi que son **T.D.O.L**.
2) Donner le résultat retourné par le module **Construction** pour les valeurs suivantes de **ch** :

ch	Résultat du module
"GOOD LUCK"	
"I AM BRILLIANT"	

3) En déduire le rôle du module.

   .....

4) Compléter le programme principal donné ci-dessous par un appel au module **Construction**.
   - **ch** est une chaîne **alphabétique majuscule** dont la taille ne dépasse pas 30 caractères.
   - **res** est le résultat de l'appel du module **Construction**.

```
Algorithme Programme_Principal
Début
  Saisie(ch)

  ..................................................................
  Afficher(res)
Fin
```

T.D.O.G

Objet	Type

Exercice 2 (3 Points)

N personnes se défient entre-elles durant les éliminatoires d'un tournoi de jeu d'échecs.

Lors de l'inscription, chaque joueur obtient le numéro qu'il porte durant le tournoi.

L'algorithme ci-dessous permet de mélanger les numéros des joueurs, puis d'afficher ceux qui vont s'affronter dans chaque partie.

```
Algorithme Tournoi_Échecs
Début
  // Étape 1 - Saisir le nombre de joueurs
  Écrire("Nombre de joueurs ? ") ; Lire(n)
  // Étape 2 - Mélanger les numéros des joueurs
  Mélanger(tj, n)
  // Étape 3 - Afficher les parties
  Afficher(tj, n)
Fin
```

TDNT
tab = tableau de 100 entier

T.D.O.G

Objet	Type
n	Entier
tj	Tab
Mélanger Afficher	Procédure

```
Procédure Mélanger(@t: tab, nj: entier)
Début
  Pour i de 0 à nj-1 Faire
    t[i] ← i+1
  Fin Pour
  Pour i de 0 à nj-1 Faire
    p ← aléa(0, nj-1)
    Permuter(t[i], t[p])
  Fin Pour
Fin
```

T.D.O.L

Objet	Type
i, p	Entier
Permuter	Procédure

1) Identifier les **types de paramètres** du module **Mélanger** dans les algorithmes ci-dessus :
   - Souligner les **paramètres formels**.
   - Encadrer les **paramètres effectifs**.
2) Retrouver les **paramètres** du module **Mélanger** ainsi que le **mode de passage** de chacun d'eux.

Paramètre	Mode de passage

3) Transformer l'**étape 1** en un module. **Effectuer le contrôle de saisie** :
   - n doit être un nombre pair
   - n ∈ [6, 100]

.....

.....

.....

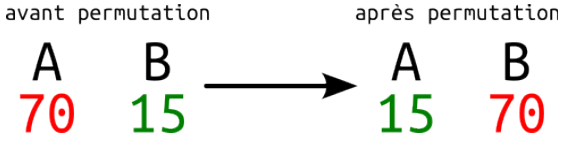
.....

.....

.....

.....

4) Développer l'algorithme du module **Permuter** qui permet d'échanger le contenu des variables passées en paramètres.



.....

.....

.....

.....

.....

.....

.....

Exercice 3 (3 Points)

Soit la séquence suivante, **n** étant un entier strictement positif :

```
s ← 0
i ← n - (n + 1) mod 2
Tant Que i > 0 Faire
  s ← s + i
  i ← i - 2
Fin Tant Que
Écrire(s)
```

n = 5

s	i	i > 0

n = 8

s	i	i > 0

1) **Retrouver la valeur affichée** par cette séquence pour les valeurs ci-dessous. **Donner la trace d'exécution.**

n	s
5	
8	

2) La séquence ci-dessus présente un choix inapproprié de la structure itérative utilisée (**Tant Que … Faire**). **Récrire cette séquence** en utilisant une autre structure plus adéquate.

.....

.....

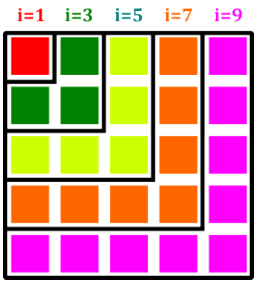
.....

.....

.....

.....

3) Sachant que la séquence ci-dessus calcule la **somme des nombres impairs inférieurs ou égaux à n**, trouver une expression simplifiée qui permet de retrouver **s** en fonction de **n**, sans utiliser de boucle.




```
fonction somme_impairs(n: entier):entier
Début
  s ← …………………………………………………………
  Retourner s
Fin
```

Problème (10 Points)

Un laboratoire de recherche effectue des expériences scientifiques et enregistre les résultats dans un tableau :

- 0 : Si l'expérience échoue.
- 1 : Si l'expérience est en cours.
- 2 : Si l'expérience a réussi.

On se propose d'écrire un programme qui permet de :

- Saisir le nombre d'expériences **nx**, (2 ≤ **nx** ≤ 100)
- Remplir un tableau **te** par les résultats de ces expériences.
  te[i] ∈ {0, 1, 2}.
- Calculer le nombre d'expériences réussies (**nr**), en cours (**nc**) et échouées (**ne**).
- Calculer le rendement du laboratoire.
  Sachant que le rendement du laboratoire est considéré comme :
  - **Bon** : Si $nr \geq \frac{75}{100} \times nx$.
  - **Satisfaisant** : Si $nr \geq \frac{50}{100} \times nx$ et $nc \geq ne$.
  - **Insuffisant** : autrement.

Exemple : Pour **nx = 11**, et les données suivantes enregistrées dans le tableau **te** :

te	2	2	1	0	0	1	1	0	2	2	0
	0	1	2	3	4	5	6	7	8	9	10

Le programme affichera :

```
Le nombre d'expériences réussies est 4
Le nombre d'expériences en cours est 3
Le nombre d'expériences échouées est 4
Le rendement du labo est insuffisant.
```

Travail demandé

1. Écrire un algorithme modulaire du programme principal.
2. Écrire l'algorithme de chacun des modules envisagés.

Réponse au Problème